



# **IOT-BASED TRUCK OVERLOAD DETECTION AND GPS-ENABLED ALERT SYSTEM**



## **A MINI PROJECT REPORT**

*Submitted by*

**THAMOHARINI K M (71812302242)**

**UMESH KARAN B (71812302248)**

**VITHYA SHREE G G A (71812302259)**

*In partial fulfilment for the award of the degree of*

**BACHELOR OF ENGINEERING**

**IN**

**ELECTRONICS AND COMMUNICATION ENGINEERING**

**SRI RAMAKRISHNA ENGINEERING COLLEGE**

**[Educational Service: SNR Sons Charitable Trust]**

[Autonomous Institution, Reaccredited by NAAC with 'A+' Grade]

[Approved by AICTE and Permanently Affiliated to Anna University, Chennai]

[ISO 9001:2015 Certified and All Eligible Programmes Accredited by NBA]

VATTAMALAIPALAYAM, N.G.G.O. COLONY POST,

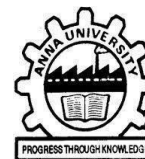
**COIMBATORE – 641 022**

**ANNA UNIVERSITY: CHENNAI-600025**

**APR - 2025**



**SRI RAMAKRISHNA ENGINEERING COLLEGE**



**BONAFIDE CERTIFICATE**  
**20EC280- MINI PROJECT II**

Certified that this 20EC280 - Mini Project II Report “**IOT-BASED TRUCK OVERLOAD DETECTION AND GPS-ENABLED ALERT SYSTEM**” is the bonafide work of **THAMOHARINI K M (71812302242), UMESH KARAN B (71812302248), VITHYA SHREE G G A (71812302259)** who carried out the project under my supervision.

**SIGNATURE**

**Dr.M.JAGADEESWARI,M.E.,Ph.D**

**Professor**

**Supervisor**

Department of ECE  
Sri Ramakrishna Engineering College  
Coimbatore - 641022

**SIGNATURE**

**Dr.M.JAGADEESWARI,M.E.,Ph.D,**

**Professor**

**Head of the Department**

Department of ECE  
Sri Ramakrishna Engineering College  
Coimbatore - 641022

**Submitted for the Mini Project Viva-Voce Presentation held on \_\_\_\_\_**

**INTERNAL EXAMINER**

**EXTERNAL  
EXAMINER**

## ACKNOWLEDGEMENT

We have immense pleasure in expressing our wholehearted thankfulness to **Shri.R.Sundar**, Managing Trustee, SNR Sons Charitable Trust and Shri. S. Narendran, Joint Managing Trustee, SNR Sons Charitable Trust for giving us the opportunity to study in our esteemed college and for very generously providing more than enough infrastructural facilities for us to get molded as a complete engineer.

We wish to express our profound and sincere gratitude to **Dr. A. Soundarrajan**, Principal, for inspiring us with his Engineering Wisdom. We also wish to record our heartfelt thanks for motivating and guiding us to become Industry ready engineers with inter- disciplinary knowledge and skillset with her multifaceted personality.

We extend our indebted thankfulness to **Dr.M.Jagadeeswari, M.E.,Phd., Professor and Head**, for guiding and helping us in all our activities to develop confidence and skills required to meet the challenges of the industry. We also express our gratitude for giving us support and guidance to complete the project duly.

We owe our deep gratitude to our project supervisor **Dr.M.Jagadeeswari, M.E.,Phd.**, of Electronics And Communication Engineering who took keen interest in our project work and guided us all along with all the necessary inputs required to complete the project work.

We express our sincere thanks to our project coordinator **Mr. M. Selvaganesh Assistant Professor** the evaluators teaching faculty members and supporting staff members of the department for evaluating the project and providing valuable suggestions for improvements.

## **ABSTRACT**

One of the major issues arising in the cargo transportation sector in roadways is that truck overloading increases the risk of accidents, rises fuel consumption, and puts extra strain on vehicles. Many trucks run either overloaded or underloaded despite legal weight restrictions, since there is no real-time monitoring mechanism in place. The inability of current techniques—such as weigh stations and human inspections—to continuously track a truck's weight during its voyage makes them inadequate. The real-time cargo weight monitoring system described is intended to improve vehicle safety and streamline logistics processes. The device gives drivers and officials instant alerts, identifies overloading or imbalances, and counts cargo weight properly. GPS tracking is also incorporated to guarantee safe transit and adherence to weight restrictions. By avoiding overload, the suggested technique decreases fuel waste, increases vehicle longevity, and lessens the likelihood of accidents brought on by incorrect weight distribution. Additionally, it increases the logistics industry's efficiency, guaranteeing safer roadways and improved adherence to regulations. By lowering costs, raising operational standards, and promoting a more sustainable approach to cargo management, the deployment of such a system has the potential to revolutionize freight transportation. This solution reduces the risks connected with overloaded trucks while enhancing the industry's reputation.

## TABLE OF CONTENTS

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>1. Introduction.....</b>                            | <b>1</b>  |
| <b>1.1. Problem Statement .....</b>                    | <b>1</b>  |
| <b>1.2. Scope .....</b>                                | <b>1</b>  |
| <b>1.3. Objective.....</b>                             | <b>2</b>  |
| <b>1.4. Overview.....</b>                              | <b>3</b>  |
| <b>2. Literature Survey .....</b>                      | <b>4</b>  |
| <b>2.1. Literature Survey - 1.....</b>                 | <b>4</b>  |
| <b>2.2. Literature Survey - 2.....</b>                 | <b>5</b>  |
| <b>2.3. Literature Survey - 3.....</b>                 | <b>7</b>  |
| <b>2.4. Literature Survey - 4.....</b>                 | <b>8</b>  |
| <b>2.5. Literature Survey – 5 .....</b>                | <b>9</b>  |
| <b>2.6. Existing Technologies.....</b>                 | <b>11</b> |
| <b>2.7. Summary .....</b>                              | <b>13</b> |
| <b>3. Methodology.....</b>                             | <b>14</b> |
| <b>3.1. Proposed System .....</b>                      | <b>14</b> |
| <b>3.2. System Overview .....</b>                      | <b>14</b> |
| <b>3.3. Block Diagram .....</b>                        | <b>16</b> |
| <b>3.4. Flow Chart.....</b>                            | <b>18</b> |
| <b>3.5. Work Flow.....</b>                             | <b>22</b> |
| <b>4. Software &amp; Hardware Implementation .....</b> | <b>24</b> |
| <b>4.1. Software Description .....</b>                 | <b>24</b> |
| <b>4.2. Hardware Description.....</b>                  | <b>25</b> |
| <b>4.3. Implementation and Calculation .....</b>       | <b>29</b> |
| <b>4.4. Blynk Implementation.....</b>                  | <b>31</b> |
| <b>4.5. Program Code .....</b>                         | <b>33</b> |
| <b>5. Experimental Results .....</b>                   | <b>52</b> |
| <b>5.1. Hardware Setup.....</b>                        | <b>52</b> |
| <b>5.2. Output.....</b>                                | <b>53</b> |
| <b>5.3. Output Description.....</b>                    | <b>54</b> |

|                                             |           |
|---------------------------------------------|-----------|
| <b>6. Conclusion and Future Scope .....</b> | <b>57</b> |
| <b>6.1. Conclusion .....</b>                | <b>57</b> |
| <b>6.2. Future Scope .....</b>              | <b>58</b> |
| <b>References .....</b>                     | <b>60</b> |

# **1. INTRODUCTION**

## **1.1. PROBLEM STATEMENT**

Overloaded trucks are a big problem in the transportation world. Not only do they put everyone on the road at risk, but they also cause serious damage to public roads, lead to mechanical breakdowns, and burn more fuel than necessary. Even though there are rules in place to prevent overloading, enforcing them is still a major challenge. The usual systems like weighbridges and random checks might have worked before, but they simply can't keep up with today's challenges. They don't offer continuous monitoring, and because they're fixed in one place and rely heavily on manual checks, it's easy for drivers to avoid them. Another issue that often gets overlooked is how the load is distributed inside the truck. If it's not balanced properly, the vehicle becomes unstable, especially during sharp turns or sudden stops. This greatly increases the risk of accidents. That's why our project focuses on building a smart, real-time solution. We're developing an IoT-powered load monitoring system that brings together load sensors, microcontrollers, wireless communication, and GPS. The goal is to track truck loads continuously, identify if the weight is overloaded or unevenly distributed, and instantly alert both the driver and the authorities when there's a problem. With this system, we hope to make our roads safer, help companies follow the rules more easily, and improve how logistics operations are managed—all through quick alerts and smart, data-driven insights.

## **1.2. SCOPE**

This project aims to create a simple, reliable, and real-time system that helps monitor the weight and balance of loads in trucks using just an ESP32 microcontroller. By combining load sensors, GPS tracking, and wireless communication via the Blynk app, the system gives both drivers and fleet managers the tools they need to stay informed about the load status at all times.

It is designed to be helpful for industries like logistics, construction, mining, and delivery services where truck overloading and uneven cargo distribution can cause major safety and financial issues. The system automatically alerts users if there's too much weight or if the cargo is unbalanced +before it becomes a serious problem.

Because it's built on widely available, low-cost components, it's easy to set up and can be expanded with new features in the future, like cloud data storage, detailed analytics, or even AI-based recommendations.

### 1.3. OBJECTIVE

**Continuous Weight Monitoring:** Mount four strain-gauge load cells under the cargo platform to monitor real-time weight data at high accuracy.

**Embedded Processing:** Connect load cells with a microcontroller (ESP32 or Arduino) to digitize measurements, calibrate readings, and calculate real-time cargo mass.

**IoT Connectivity:** Leverage ESP32's in-built Wi-Fi, LoRa, or GSM connectivity to periodically transmit weight information with low latency to a cloud server.

**Cloud Data Management:** Store incoming data on a scalable IoT platform (Blynk), allowing for continuous logging, trend analysis, and secure access controls.

**Automated Alerting:** Set up threshold checks on the microcontroller and/or cloud backend to cause onboard alarms (buzzer/LCD) and remote notifications when weights surpass legal or user-defined thresholds.

**Remote Dashboard:** Offer fleet managers, drivers, and regulators a web/mobile-responsive interface for real-time visualization, historical replay, and exportable reports.

**Regulatory Compliance & Safety:** Minimize infrastructure stress, decrease accident likelihood, and provide weight regulation compliance through

active overload detection.

**Scalability & Extensibility:** Develop modular hardware and software such that further sensors (GPS, temperature, vibration) or advanced analytics (predictive load balancing) can be added in future releases.

## 1.4. OVERVIEW

The focus of this project is to develop a smart Truck Cargo Weight Monitoring and Alert System that ensures balanced loading and avoids overloading of commodities in commercial transport trucks. The system makes use of load sensors positioned strategically to monitor weight imbalance on the cargo platform. Whenever an imbalance or an overload is identified, the system sends an automatic alert.

In contrast to previous systems that used SMS notification through SMTP, the project uses Blynk IoT to provide instant pop-up alerts on the user's phone directly. The messages are fast, concise, and not cellular SMS service-based, which makes the system efficient and current.

In addition to overload detection and alarm, the project also supports integration of NEO-6M GPS modules for real-time location tracking of trucks. The data is shown in the Blynk app to provide both load condition monitoring and location awareness in a single screen. The programming is through the Arduino IDE, and some of the hardware devices used include load cells, weight amplifiers, GPS modules, and ESP32 for Blynk IoT connectivity.

By integrating hardware sensors, cloud IoT solutions, and mobile interaction, this project improves safety, enforces road rules, and offers an intelligent logistics solution for the transport industry.

## **2. LITERATURE SURVEY**

### **2.1. LITERATURE SURVEY – 1**

This paper discusses the increasing problem of truck overloading and suggests an embedded system-driven solution to detect and counter it effectively. Commercial truck overloading is a serious problem in road transport because it adversely affects road infrastructure, leads to more mechanical failures, decreases vehicle efficiency, and increases the risk of accidents. Conventional techniques for detecting overloads are dependent on manual weighing at roadblocks, which are not time-efficient, discretionary, and ineffective. To counter these difficulties, the authors developed a real-time embedded system to automatically sense overloading conditions in trucks based on microcontroller technology.

The system utilizes various important hardware components that cooperate seamlessly. Load cells are installed on the truck chassis to weigh the load being transported. The load cells transform the mechanical force exerted by the truck's cargo into analog electrical signals. The signals are then passed through an Analog-to-Digital Converter (ADC) to digitize them, which are then processed by a microcontroller. The microcontroller continuously monitors the incoming data against a predefined legal load threshold. If the weight is higher than the threshold, the microcontroller activates warning signals, like a buzzer and LED lights, to notify the driver regarding the overload condition. Aside from the local warning system, the embedded system also has a Global System for Mobile Communication (GSM) module that can send SMS warnings to fleet managers or authorities, giving real-time updates on the state of overload. Moreover, the Global Positioning System (GPS) module is also integrated to monitor the location of the truck. This integration allows the remote tracking of the route and location of the truck, providing useful information on route optimization, compliance verification, and legal records.

The system was designed with low-cost deployment in mind to make it

available to small and medium-sized transport companies. The system is also very scalable and modular, enabling customization to fit various truck types and capacity capacities. The prototype in experimentation showed that it could detect overloads precisely and rapidly, triggering alarms in milliseconds of an overload occurring. This system is thus a useful and dependable solution for controlling overloading in actual logistics operations.

**Output:** The system effectively sensed overloading situations and triggered alerts through LED indicators and a buzzer. Real-time SMS alerts were sent to registered numbers via the GSM module. GPS information was sent accurately, displaying the vehicle's location at the time of overloading. Ongoing testing proved steady operation and dependable performance under disparate loads.

## 2.2. LITERATURE SURVEY – 2

This paper introduces a novel optimal system design for Weigh-In-Motion (WIM) systems employing in-pavement strain sensors for dynamic monitoring and measurement of vehicle loads traveling on roadways. Traditional WIM systems are often accuracy-constrained and installation- and maintenance-cost-prohibitive. In order to reduce these problems, the authors develop a system strategy with cost-effectiveness and measurement accuracy in mind with integrated strain sensors within the pavement.

The research systematically analyzes various sensor configurations, pavement response, and vehicle dynamics models in an effort to find the optimum sensor locations and system parameters. Since the sensors are embedded in the pavement itself, the system can continuously measure the pavement's strain response as vehicles pass through, which is then used to estimate the accompanying axle loads. The key innovation involves the optimization of sensor layout geometry and signal processing algorithms, which minimizes the error of weight estimation.

Large-scale simulation and field testing were conducted to evaluate the performance of the system. The results showed that the proposed system significantly improved the accuracy of vehicle weight estimation, even at varying speeds and axle configurations. Furthermore, the use of strain sensors enables non-intrusive and durable means, thereby supporting the possibility for long-term use on highways as well as smart transportation systems. Moreover, except for load estimation, the collected data by the WIM system may also be used for monitoring traffic, health checks of infrastructures, as well as the enforcement of weight regulations. The continuous and real-time capability of the system to supply data is conducive to the overall objectives of intelligent transportation systems (ITS) and smart infrastructure. In general, the study introduces a major innovation in transportation monitoring by showing how in-pavement sensor technology, coupled with system design optimization, can yield more efficient and dependable WIM solutions with uses in logistics, infrastructure management, and road safety.

**Output:** The in-pavement proposed strain sensor system precisely quantified vehicle axle loads in real time. Optimized sensor positioning and signal processing also greatly decreased estimation errors. Field tests validated the system's reliability and performance under different traffic conditions. The solution aids intelligent transportation through facilitating effective, uninterrupted weight measurement.

### **2.3. LITERATURE SURVEY – 3**

The authors propose a novel approach to real-time volume estimation using a LiDAR sensor mounted on an excavator. The 3D point cloud generated from the LiDAR enables accurate measurement of soil volume loaded into trucks. Using techniques like Delaunay triangulation and MATLAB's alpha shape function, the volume is calculated with an error margin of less than 2%. This method eliminates disputes between suppliers and customers and enables better

scheduling and load monitoring. Although effective, the solution requires high setup costs and depends on optimal environmental conditions.

The authors offer a practical and well-engineered solution for determining truck load volume using a LiDAR system integrated into the boom of an excavator. Their system is capable of providing highly accurate volume readings in real time—something that's incredibly valuable at busy construction sites. It removes the need for manual volume estimation and helps operators avoid overloading or underloading. Plus, the real-time visibility of the load helps both the operator and truck driver monitor the loading process effectively. On the technical side, it uses point cloud data processed through Delaunay triangulation and MATLAB, which is known for precision. However, the setup requires sophisticated hardware and might be costly for small-scale use. Also, performance may dip under extreme weather conditions. Overall, the paper presents a solid, field-ready approach that aligns well with modern construction site needs.

## **2.4. LITERATURE SURVEY – 4**

This research work tackles the sophisticate bulk port belt-conveyor routing problem with the aim of combining storage areas and loading operations into the conveyor routing system. In this research work, the authors recognize that the conventional methods of conveyor routing tend not to address the dynamic and interdependent factors of storage availability and the loading processes, which may result in inefficiency and delays in bulk port operations. To tackle this problem, the authors introduce a new optimization model that considers these interactions with the goal of enhancing the overall performance and efficiency of bulk port logistics. The suggested model considers a number of important factors, including the location and availability of storage facilities, loading capacities, and the throughput capacity of the conveyor belt system. By taking all these factors into account collectively, the model facilitates routing decisions that are more

accurate and efficient and that result in reduced congestion and costs of operation. The paper's significant contribution is the feature of dynamically adapting to the changes in real-time, where the levels of storage vary and the loading operations must be coordinated with the availability of conveyor belts.

The authors adopt a mathematical optimization method along with simulation methods for modeling and solving the problem. The model supports improved decision-making in port operation by optimizing the flow of materials, timing, and utilization of resources. The paper demonstrates multiple experiments and findings, illustrating that the model vastly enhances operational efficiency and minimizes bottlenecks compared to conventional routing strategies. In addition, the paper also addresses the scalability of the model, with the implication that it can be adapted to other bulk port systems with diverse operational constraints. The solution presented opens the eyes of readers to understanding how optimization approaches can be applied to enhance the handling of conveyor routing in ports, with emphasis on real-time decision-making and synchronization of resources.

**Output:** The model to be used optimized conveyor routing by combining storage and loading interactions. It minimized material handling delays and enhanced throughput in bulk port operations. The system evolved dynamically according to real-time updates in storage space availability and load schedules. It was scalable and worked effectively in optimizing the efficiency of port logistics networks.

## 2.5. LITERATURE SURVEY – 5

This work presents a new approach to real-time payload measurement on a lorry platform via a 3D LiDAR sensor system installed on the boom of an excavator. Conventional methods of payload measurement are usually mechanical sensors or post-operation weighing, which are cumbersome and not responsive in real time. To overcome these issues, the authors created a system

that utilizes 3D point cloud information from a LiDAR sensor to estimate the volume and then the weight of the material loaded on the truck. The prime benefit of the system is in its capability to record the loaded material's shape and distribution as it occurs in the actual process of excavation and loading. High-resolution spatial information of the truck platform is taken by the 3D LiDAR before and after loading, enabling one to precisely determine the volume change. Using the material density values, the system makes real-time estimation of the payload without requiring human intervention or auxiliary weighing instruments. The research gives a detailed description of the data processing pipeline, from point cloud filtering, ground surface estimation, to volume modeling, as well as dynamic adjustments for vehicle motion and tilt. Mounting the LiDAR on the boom of the excavator allows the sensor to have a free view overhead, with a good data capture point throughout the loading process. Experimental validation was done in field conditions for the construction site and mines, and results showed that the system could attain high accuracy for payload estimation and that errors were mostly within the acceptable limit for operational decision-making. The system also offers operational efficiency, minimizes the risk of overloading, and enhances logistics and material flow monitoring in construction and mining use. This technique demonstrates how advanced sensing technologies can be integrated into automated heavy machinery systems, enabling smart data-driven operation in infrastructure and logistics industries. The approach provides future scope for complete autonomy in loading and transport monitoring systems.

**Output:** The integrated 3D LiDAR system provided accurate real-time estimations of payload volume during loading. It estimated material weight from point cloud data scanned and known densities. The system performed effectively without disturbing excavation activities or demanding additional hardware. Field tests verified the reliability of the method for increasing material tracking and transport safety.

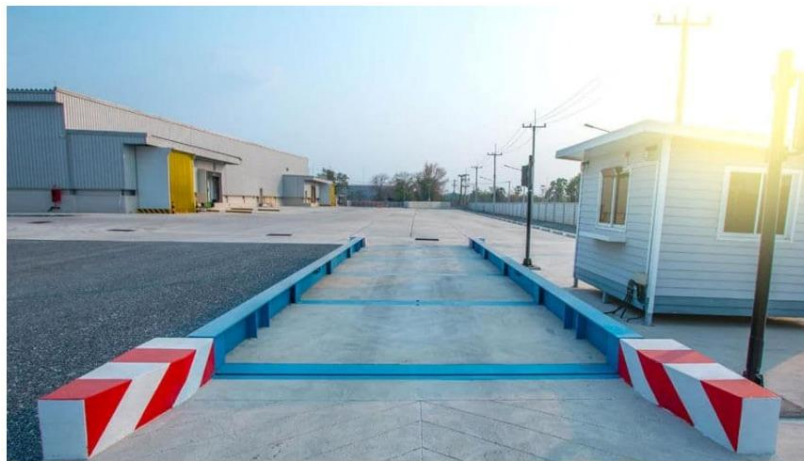
## 2.6. EXISTING TECHNOLOGIES

### Static Weighing Systems

Trucks must stop at specified weigh stations, where weights of axles are recorded from large, mounted platform scales set in the ground. Total weight is computed manually or electronically from these measurements.

Need trucks to stop or drive over embedded sensors; not real-time; costly to install and maintain; restricted to installed locations.

Our system is mounted on a vehicle and provides real-time weight monitoring while in motion—no stopping or external infrastructure required.



### Weigh-In-Motion (WIM) Systems

WIM systems employ road-mounted sensors to weigh a truck in motion. They are typically installed at toll plazas or highway stretches to identify overweight vehicles without having to stop them.



WIM systems are too expensive to be embedded in the road, only work at one location, are sensitive to road conditions and speed of vehicles, and hence not appropriate for route-wide real-time overload monitoring.

Our system is mounted on the vehicle itself and hence is portable and uniform in any location and road condition. It minimizes reliance on infrastructure embedded in the road and also decreases costs.

### **LiDAR-Based Load Volume Measurement**

LiDAR sensors compute the material volume loaded in a truck, and the system estimates the weight using the material density known beforehand.

LiDAR load volume measurement is based on 3D laser sensors, which are sensitive to material reflectivity, weather, and sensor position. It also depends on precise material density to calculate weight, which might not be accurate for every type of cargo. LiDAR systems are also costly and may not fit every vehicle.

Our system measures the actual weight directly, as opposed to estimating based on volume, providing greater reliability for all types of cargo. It is easier to install and does not depend on external material data.



## **2.7. SUMMARY**

Static Weighing, WIM, and LiDAR-based systems are limited by the fact that they necessitate stopping, are location-dependent, or need to use material density in estimation of weight. Our mobile system gives real-time accurate weight tracking without stopping, offering portability, consistency, and increased reliability, sidestepping the shortcomings of the traditional systems.

## **3. METHODOLOGY**

### **3.1. PROPOSED SYSTEM**

- The proposed system addresses limitations of existing systems by implementing a real-time cargo monitoring solution using IoT. Key components of the system include:
- Load Sensors (e.g., strain gauge-based load cells): Installed on the vehicle platform to measure cargo weight.
- Microcontroller (e.g., Arduino or ESP32): Collects data from sensors and processes it.
- IoT Module (e.g., Wi-Fi/LoRa/GSM): Transmits data to the cloud.
- Cloud Platform (e.g., ThingSpeak, Blynk, Firebase): Stores, analyzes, and displays weight data.
- Mobile/Web Dashboard: Provides a user interface for fleet managers and authorities.
- Overload Alert System: Sends SMS/email/push notification if weight exceeds safe limit.
- This system continuously tracks cargo weight and automatically notifies stakeholders if an overload is detected, helping in quick decision-making and preventive action.

### **3.2. SYSTEM OVERVIEW**

Managing truck loads and ensuring safety doesn't have to be complicated—and that's exactly what our updated Real-Time Load Detection and Alarm System is all about. Designed around the powerful and flexible ESP32 microcontroller, this system brings everything together in one compact setup, cutting down complexity while boosting performance.

#### **Smart Sensing**

At the core of the system are HX711 load cell amplifiers connected to load cells placed on the truck bed. These sensors constantly keep an eye on the cargo weight. As the load changes, the sensors send analog signals to the HX711, which amplifies and converts them into digital form so the ESP32 can make sense of the data.

## **ESP32**

The ESP32 acts as the the all-in-one brain of the system. It handles both the number crunching and communication duties. Once it receives the weight data from the HX711, it checks whether the truck is overloaded or if the weight isn't evenly distributed. If anything's off, the ESP32 triggers a buzzer right on the truck to alert the driver and also sends a notification straight to a smartphone using the Blynk IoT app.

## **Real-Time Tracking with GPS**

To take things a step further, the system also includes a NEO-6M GPS module. This adds real-time location tracking, so managers or operators can know exactly where the truck is at any moment. The ESP32 collects the GPS data and pushes it to the Blynk dashboard, where it's all displayed in an easy-to-use interface.

From cargo weight to system alerts and location tracking—it's all right there in the Blynk app. Whether you're a fleet manager sitting in the office or a driver on the road, this setup keeps you informed and in control, anytime, anywhere.

This updated system is not only compact and cost-effective, but it's also built to scale. Whether you're managing one truck or an entire fleet, it's a smart and reliable solution for improving safety and logistics efficiency.

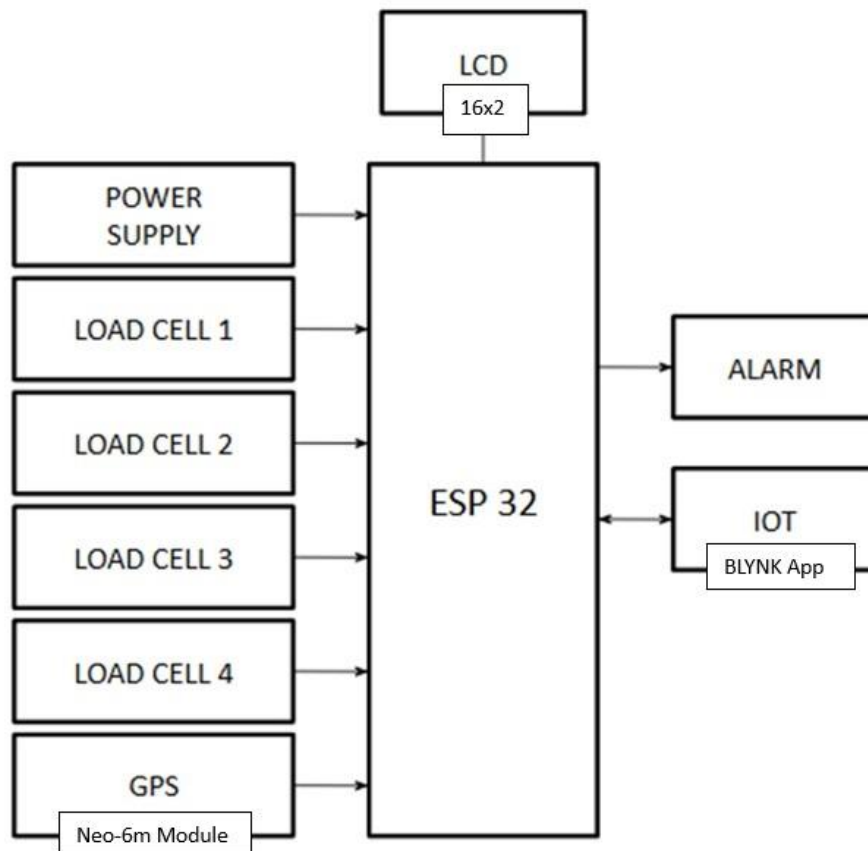
### **3.3. WORKING PRINCIPLE**

The real-time load detection and alarm system operates by integrating live weight sensing, GPS tracking and wireless communication into a single smooth workflow all driven by the esp32 microcontroller. first the 5kg plastic load cells sense any weight applied to the truck bed and generate very small electrical signals these are too small to be read directly so they are amplified through a special load cell amplifier module after amplification the information is input into the esp32 the esp32 continuously reads this data and processes it in real time if the system finds that the weight has exceeded a predetermined safe limit or that the load is unevenly distributed on the platform it instantly activates a buzzer to inform the driver present simultaneously the esp32 pushes a notification to a smartphone that is connected through the blynk iot platform this means that even remote users or fleet managers are instantly notified about the condition of the vehicle also the neo-6m GPS module delivers real-time location information which is refreshed and shown on the blynk dashboard along with the weight status overall the system provides an assured real-time means of monitoring and reacting to hazardous loading conditions preventing accidents safeguarding vehicles and enhancing overall transportation safety

### **3.4. BLOCK DIAGRAM**

The functional design of an embedded component and an Internet of Things (IoT) connectivity-based real-time truck load monitoring and alert system for safe and effective cargo transportation is shown in the block diagram. The ESP32 microcontroller serves as the central processing unit and was chosen because of its robust processing capability and built-in Wi-Fi module, which enable seamless communication with Internet of Things platforms. The entire system runs on a controlled power source that guarantees constant voltage levels for all linked devices, including sensors, display modules, and communication

modules.

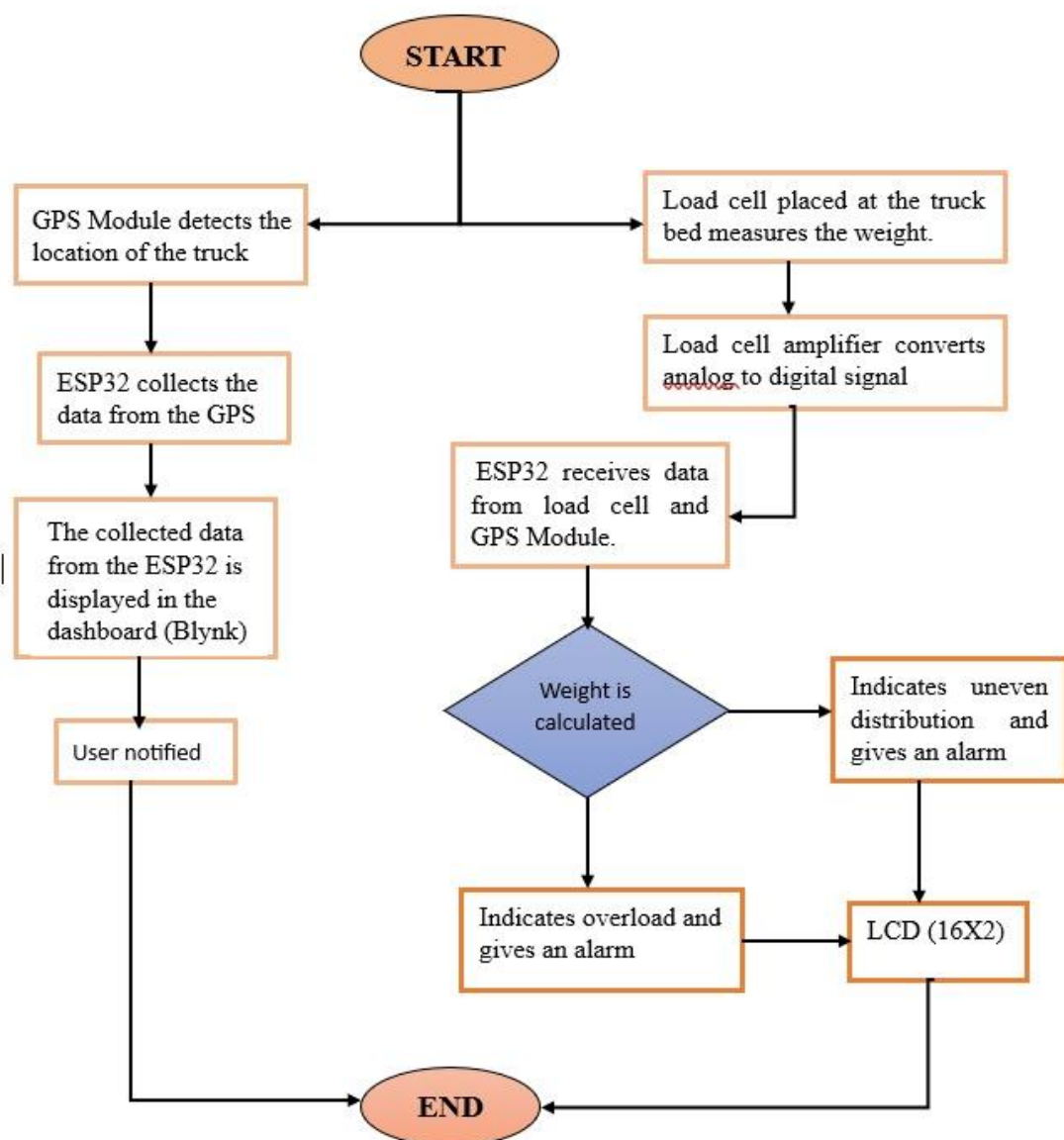


Four 5 kg plastic load cells that are installed underneath the truck's cargo platform are part of the system. These load cells provide accurate weight data by measuring the distributed load at four different points. Every load cell transforms the cargo's mechanical force into an electrical signal. Before the signals reach the microcontroller, they are either enhanced and digitalised by signal amplifiers like the HX711 or read directly by the ESP32 (if pre-calibrated modules are being used). The ESP32 then calculates the total load being carried after processing the incoming weight data and calibrating it against a reference. Overloading and load imbalance are two important factors in preventing accidents and road damage, and this real-time data aids in their detection.

The system has a NEO-6M GPS module that connects to the ESP32 and gives the vehicle's current location in addition to weight monitoring. This GPS

information is essential for route mapping and pinpointing potential overloading spots. The ESP32's Wi-Fi capability is then used to send the GPS coordinates and weight readings to the Blynk IoT platform. Fleet managers, drivers, and regulators can view real-time data on their PCs or smartphones using the Blynk app, which serves as a user interface. The app provides a dashboard with widgets that show the location of the vehicle and current weight.

### 3.5. FLOW CHART



### **System Startup (Start)**

The system starts its operation upon powering on the truck or being in an active monitoring mode. All modules and sensors are started. This includes the powering up of the load cells, GPS module, ESP32 controller, LCD, and buzzer. From here on, the system continuously monitors location and weight in real time.

### **Measurement of Load by Load Cells**

Four plastic 5kg load cells are positioned under the corners of the truck bed in a strategic manner. These sensors measure vertical compression and generate small analog voltages in relation to the applied load. Having multiple sensors ensures proper measurement of total load and load distribution, which can detect not only overloading but also uneven loading that will destabilize the moving vehicle.

### **Signal Conditioning – Analog to Digital Conversion**

Because the load cells' output is analog and extremely low in voltage, the signal has to be amplified and digitized. A load cell amplifier (such as HX711) accomplishes this. The module reads the analog information and provides a digital signal that ESP32 can utilize. This is essential in keeping the weight calculation very accurate.

### **Real-Time Location Detection Using GPS**

A NEO-6M GPS module takes the coordinates of the position of the truck. It transfers latitude and longitude data through UART communication to the ESP32. This capability enables the system to not only sense overloading but also record the location at which overloading or imbalance has happened, which can be significant for regulatory purpose and route analysis.

### **Acquisition of Data by ESP32 Microcontroller**

The ESP32 acts as the center of the system. It reads digital signals of both

the HX711 modules (weight data) and GPS module (coordinate data). Based on the collected data, ESP32 carries out mathematical computations in order to determine the cumulative weight, and cross-checks against a preset statutory limit or ascertains uneven distribution of loads.

### **Decision Making – Overload or Imbalance Detection**

- After calculating the data, the ESP32 cross-references the weight with predetermined limits.
- When the total weight crosses the safe limit, it is classified as overload.
- When a corner or one side of the load cells has readings far above the others, it is identified as uneven distribution.
- Such conditions lead to very severe safety threats like tire bursts, brake failure, or rollover of the vehicle, and therefore, rapid detection is required.

### **Local Alerts – Driver Notification**

- When it senses an abnormal state, the ESP32 activates a buzzer and displays an LCD screen (16x2) installed within the vehicle.
- For overload, a message such as "Overload Detected!" is shown.
- For imbalance, a message such as "Load Uneven!" is shown.
- These warnings are intended to alert the driver promptly, enabling him to react (e.g., stop loading, redistribute cargo, or stop the vehicle).

### **Cloud Transmission through Blynk IoT**

In addition to local notification, ESP32 utilizes Wi-Fi functionality to forward data to the Blynk IoT cloud. Weight readings, alert flag, and GPS position are periodically sent or when triggered. Data shows up on the Blynk dashboard using the mobile application or web console.



## Blynk Features



### Remote Alert – Pop-Up Alert on Blynk App

Rather than applying SMS via SMTP, which must be accessed by a SIM card and carrier, the system avails itself of Blynk's push feature. In the event of overloading or overbalance, there is a pop-up notification issued to the smartphone of the user in real-time. This allows the system to be more affordable, contemporary, and immediate.

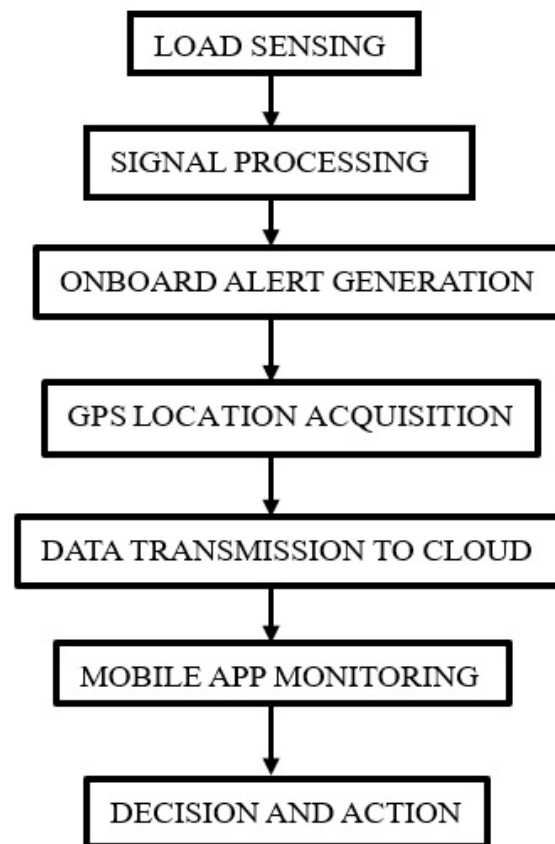
### Data Logging and Visualization

The Blynk platform also supports data visualization in the shapes of gauges, graphs, and tables. Historical weight data may be logged by the fleet operator, locations of the vehicles monitored, and even reports downloaded for auditing purposes to ensure compliance. This enhances transparency, fleet optimization, and safety management.

### Loop Continuation or System Reset (End)

Upon notification and recording, the system either loops to continue real-time monitoring or goes to standby based on vehicle movement. The loop continues as long as the truck remains in service to provide real-time surveillance continually.

### 3.6. WORKFLOW



#### 1.LOAD SENSING

This is the first stage of the system. 5kg plastic load cells are mounted beneath the cargo platform to detect weight. When a load is applied, the cells convert mechanical force into analog electrical signals. Each sensor measures weight at a specific point, allowing both total weight and distribution to be analyzed.

#### 2. SIGNAL PROCESSING

The analog signals of the load cells are transmitted to the ESP32 microcontroller, where they are digitized and processed. Calibration equations are used to translate voltage variations into precise weight measurements. The overall load is calculated by adding the readings of all four sensors.

### **3. ONBOARD ALERT GENERATION**

If the total measured weight surpasses a predetermined safety limit, local alarms are activated by the ESP32 automatically. A buzzer beeps and a warning message is displayed on the LCD, warning the driver in real time of the condition of overloading.

### **4. GPS LOCATION ACQUISITION**

Simultaneously, the NEO-6M GPS module that is attached to the ESP32 constantly retrieves the current geographic location of the truck. This GPS location information is critical for tracking in real time and is combined with weight information.

### **5. TRANSMISSION OF DATA TO CLOUD**

The ESP32 transmits both the GPS and weight information to the Blynk IoT Cloud using its onboard Wi-Fi. This provides remote access to the data at any place that has internet coverage.

### **6. MONITORING THROUGH MOBILE APP**

Data sent to the Blynk cloud is displayed on the Blynk mobile dashboard. Real-time weight from every load cell, total weight, and truck location are shown by the app. If there is an overload detected by the system, a push notification to the user is immediately sent.

### **7. DECISION AND ACTION**

On the basis of information illustrated on the app, drivers or fleet managers can respond in real time. This might involve offloading surplus goods, readjusting delivery, or routing the truck elsewhere. Long-term data can also be employed for maintenance, regulatory compliance, and logistics scheduling.

## 4. SOFTWARE AND HARDWARE IMPLEMENTATION

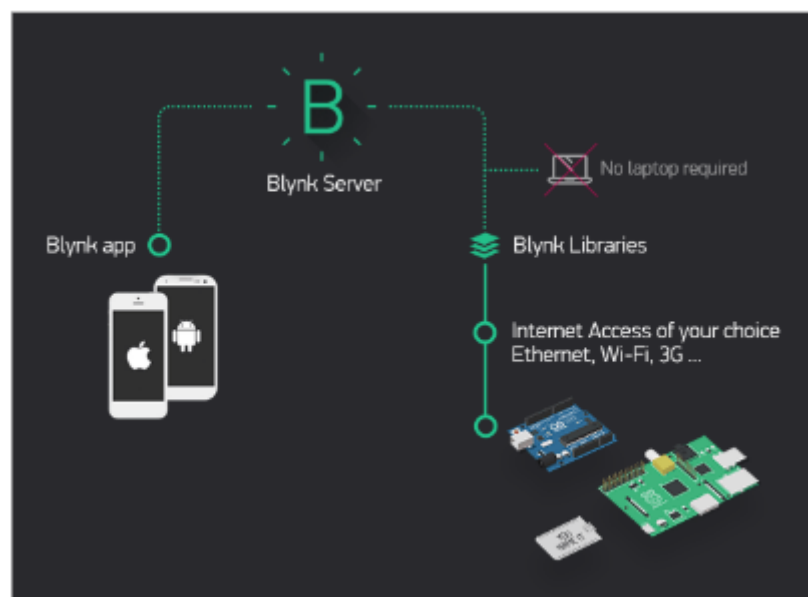
### 4.1. SOFTWARE DESCRIPTION

#### BLYNK

Blynk was designed for the Internet of Things. It can control hardware remotely, it can display sensor data, it can store data, visualize it and do many other cool things.

There are three major components in the platform:

- Blynk App - allows to you create amazing interfaces for your projects using various widgets we provide.
- Blynk Server - responsible for all the communications between the smartphone and hardware. You can use our Blynk Cloud or run your private server locally. It's open-source, could easily handle thousands of devices and can even be launched on a Raspberry Pi.
- Blynk Libraries - for all the popular hardware platforms - enable communication with the server and process all the incoming and outgoing commands.



## ARDUINO IDE

IDE stands for “Integrated Development Environment. Arduino IDE is a software program that is mainly used for editing, compiling and uploading the code in the Arduino Device. Almost all Arduino modules are compatible with this software that is an open source and is readily available to install and start compiling the code on the go. In this article, we will introduce the Software, how we can install it, and make it ready for developing applications using Arduino modules. Arduino IDE is an open-source software that is mainly used for writing and compiling the code into the Arduino Module. It is an official Arduino software, making code compilation too easy that even a common person with no prior technical knowledge can get their feet wet with the learning process.

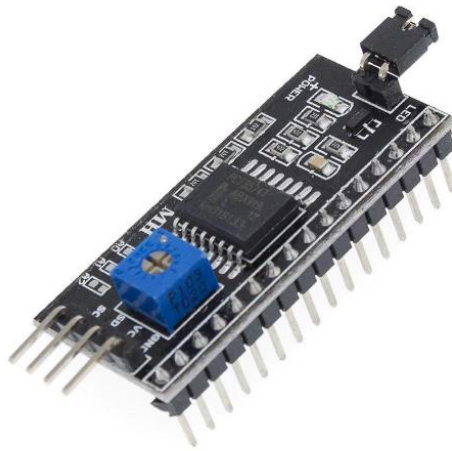


## 4.2. HARDWARE DESCRIPTION

### I2C MODULE

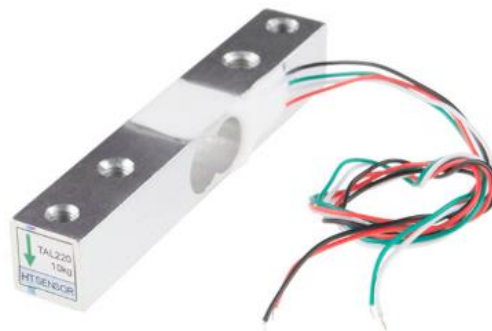
The PCF8574 is an I2C-controlled I/O expander that provides 8-bit parallel input/output (I/O) pins. It allows you to control up to 8 digital I/O pins using only two wires for communication (SDA and SCL), making it ideal for applications where GPIO (General Purpose Input/Output) pins are limited. The module is

widely used for interfacing devices like LEDs, switches, and other digital devices with microcontrollers via I2C communication.



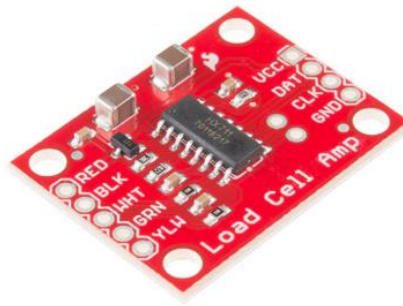
## **LOAD CELL**

A load cell is a physical element that converts pressure or force into an electric signal.



## **HX711 AMPLIFIER**

The change in the electric signal produced in a load cell is a very small change, and, for most devices, couldn't actually be detected, let alone detected accurately. So, we need another device that can either accurately measure very small changes in resistance or a device that can take that very small change in resistance and turn it into something that we can measure accurately. We are using the HX711 amplifier for that.

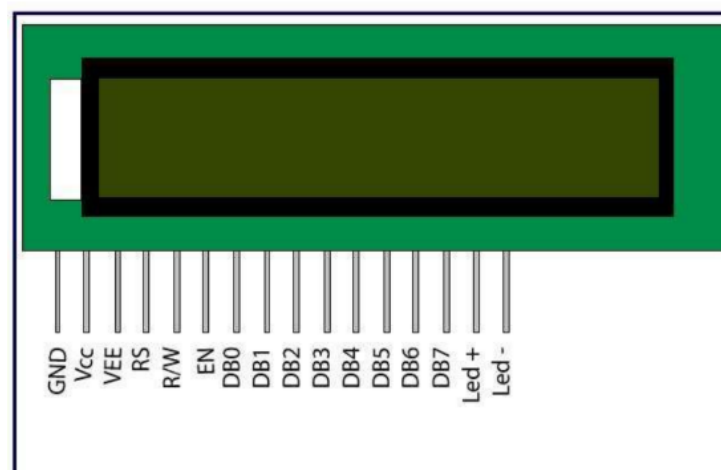


## LCD DISPLAY

LCD (Liquid Crystal Display) screen is an electronic display module and finds a wide range of applications. A 16x2 LCD display is a very basic module and is very commonly used in various devices and circuits. These modules are preferred over seven segments and other multi-segment LEDs. The reasons being: LCDs are economical, easily programmable, and have no limitation of displaying special and even custom characters (unlike in seven segments), animations, and so on.

A 16x2 LCD means it can display 16 characters per line, and there are 2 such lines. In this LCD, each character is displayed in a 5x7 pixel matrix. This LCD has two registers, namely, Command and Data.

The command register stores the command instructions given to the LCD. The data register stores the data to be displayed on the LCD. The data is the ASCII value of the character to be displayed on the LCD.



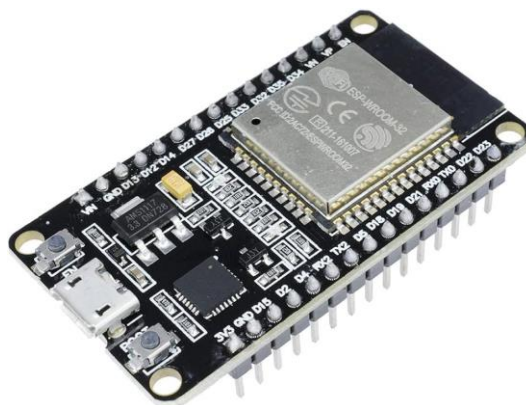
## GPS MODULE WITH PATCH ANTENNA

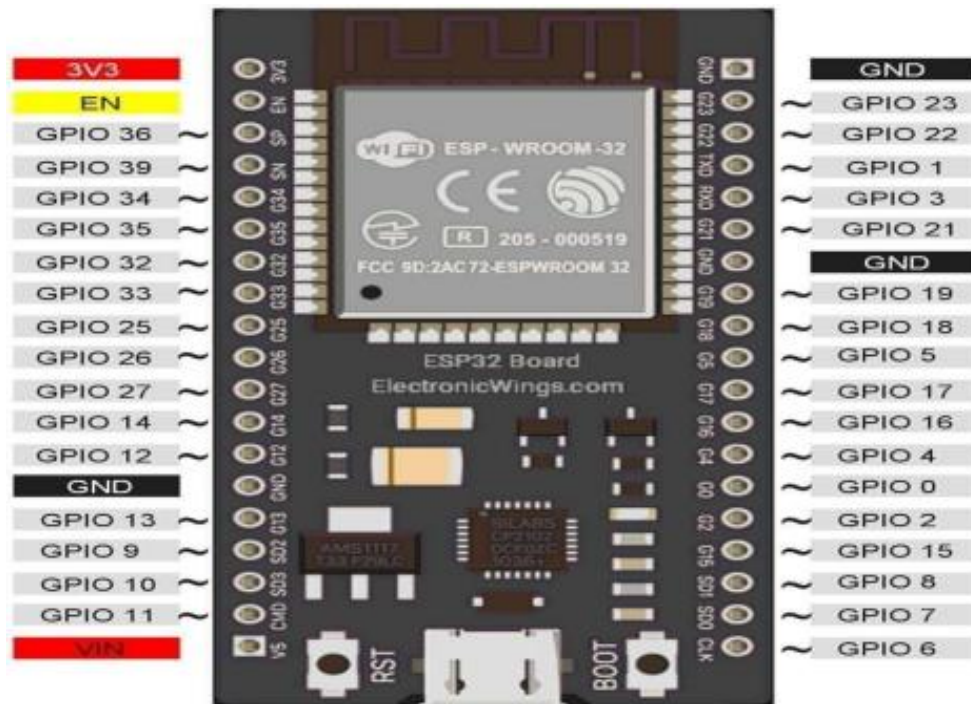
A GPS module with a patch antenna is a compact device used to determine precise location using signals from satellites. The patch antenna, which is flat and square-shaped, helps the module get a strong and stable signal—especially when placed facing the sky. It's commonly used in DIY electronics, robotics, and vehicle tracking projects because it's reliable, affordable, and easy to integrate. Think of it like giving your project a sense of direction and location awareness.



## ESP32

The ESP32 is a powerful, low-cost microcontroller with built-in Wi-Fi and Bluetooth, perfect for IoT projects. It's like a tiny brain for smart devices, capable of handling multiple tasks, connecting to the internet, and interacting with sensors and other electronics. Thanks to its speed, versatility, and energy efficiency, it's a favorite among hobbyists and developers alike.





## BUZZER

A buzzer is a small audio device that produces sound—usually a beep or tone—when powered. It's often used in electronics to give alerts, notifications, or feedback, like the beep of a microwave or an alarm system.



## 4.3. IMPLEMENATION AND CALCULATION

Connect all the hardware components by following their respective pinout structures with respect to the block diagram in section 3.2.

## Calculations

Each load cell uses an HX711 module to read the raw sensor data. The reading is then converted into weight in kilograms using a scaling factor, and sometimes a small correction factor is applied.

### Weight Calculation Formula

$$Weight_i = V_{raw(i)} \times C_i$$

Where:

- $Weight_i$  is the weight measured by the  $i^{th}$  load cell (in kilograms).
- $V_{raw(i)}$  is the raw sensor output from the HX711 module for the  $i^{th}$  load cell.
- $C_i$  is the calibration factor specific to the  $i^{th}$  load cell.
- 

### Total Weight Calculation

The total weight acting on the truck is calculated by summing the weights from all four load cells:

$$Total\ Weight = \sum_{i=1}^4 Weight_i = \{kg\}_1 + \{kg\}_2 + \{kg\}_3 + \{kg\}_4$$

### Imbalance Detection

$$Average = \frac{kg_1 + kg_2 + kg_3 + kg_4}{4}$$

#### Deviation from Average

$$diff_i = |Average - kg_i|$$

If any of  $diff_i \geq 130$ , then load is considered unbalanced

## **4.4. BLYNK IMPLEMENTATION**

### **Sign up for a Blynk Account**

First, download the Blynk app from Google Play Store or Apple App Store. After installation, launch the app and create a new Blynk account with a valid email address. This account is important because it keeps all your project information and provides access to several devices. Although you might be an experienced Blynk forum user, this is an independent account. Having an account in Blynk ensures that your projects are safely stored and can be accessed for remote monitoring and control.

### **Create a New Project**

Once you log in, you will be asked to create a new project. Click on the "New Project" button and give your project a name. This is used to recognize your work, particularly when working on several projects. You will also be required to select the device you intend to use (such as ESP8266, Arduino, NodeMCU, etc.), and the type of connection (Wi-Fi, Bluetooth, etc.), which are necessary for effective communication between the hardware and the app.

### **Select Your Hardware**

Pick the particular hardware platform you are using. Blynk supports a broad variety of hardware such as Arduino, ESP32, ESP8266, Raspberry Pi, and many others. Selecting the appropriate hardware guarantees compatibility and that the app will produce the proper code snippets and connection parameters for your device. This is a critical step for smooth communication between your mobile application and the IoT device.

### **Obtain the Auth Token**

As soon as you create a project, Blynk will automatically create an Auth Token — a special ID that bridges your app and your hardware. It's sent via email

to the address you're registered with, and you can also paste it from device settings in the application. You must insert the Auth Token into your device code as a bridge between the device and the application. Never give your token to anyone else since it grants access to your hardware controls.

### **Add and Customize Widgets**

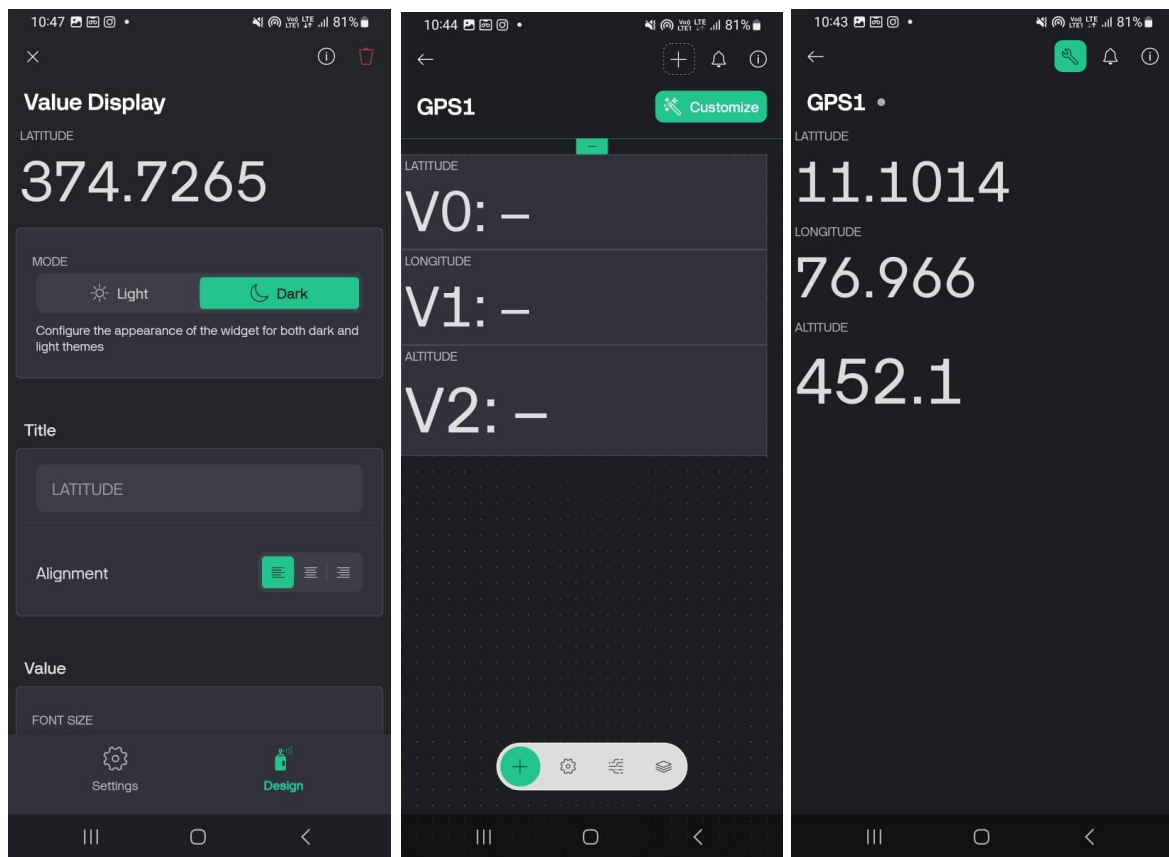
Your project screen (canvas) will be empty after having set up the fundamental project. Tap on the screen to reveal the Widget Box and choose a widget to add — for example, a button to switch on an LED. After adding, tap the widget once again to access its settings, where you can rename it, assign it to a digital/analog pin, and customize its behavior (e.g., switch or push mode). You may add several widgets such as sliders, graphs, or displays based on your project needs.

### **Organize and Test Widgets**

After adding widgets, drag and drop them to organize your interface layout. Make sure each widget is connected to the correct GPIO pin that corresponds with your hardware setup. This arrangement enhances usability and makes it easier to interact with the hardware. Testing can be done by uploading the firmware with the Auth Token to your device and observing real-time control through the app.

### **Deploy and Monitor Your Project**

After all that is in place, plug in your hardware into the internet and execute the code embedded with the Auth Token. Launch the Blynk app and initiate the project — the app will now talk to the hardware. You can control it by clicking on buttons or setting values using widgets. Observe the output or action of your device in real time, and adjust in the app interface as necessary.



## 4.5. PROGRAM CODE

```
#include <LCD_I2C.h>
LCD_I2C lcd(0x27);

#include <TinyGPS++.h>

/* Create an object named gps of the class TinyGPSPlus
*/
TinyGPSPlus gps;

volatile float minutes, seconds;
volatile int degree, secs, mins;
float latn, longn;
```

```

double lat_val, lng_val, alt_m_val;
uint8_t hr_val, min_val, sec_val;
bool loc_valid, alt_valid, time_valid;

String s = "www.google.com/maps/dir/";

#define BLYNK_PRINT Serial
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

#define BLYNK_TEMPLATE_ID "TMPL3K41_JCGw"
#define BLYNK_TEMPLATE_NAME "OVER LOAD INDICATOR"
#define                                     BLYNK_AUTH_TOKEN
"Cg6sdbfjRFD9ZLQpFriIweyCbG19lxI1"

char ssid[] = "IOT";
char pass[] = "123456789";
char auth[] = BLYNK_AUTH_TOKEN;

int buttonread;

#include "HX711.h"
const int dout1 =18;
const int sck1 = 5;

HX711 scale1;
float kg1,p1;
int flotv1;

```

```

#include "HX711.h"
const int dout12 =27;
const int sck2 = 14;

HX711 scale2;
float kg2,p2;

#include "HX711.h"
const int dout3 =13;
const int sck3 = 12;

HX711 scale3;
float kg3,p3;

#include "HX711.h"
const int dout4 =17;
const int sck4 = 16;

HX711 scale4;
float kg4,p4;
float total_kg;

#define buzzer 23
//float dkg1,dkg2,dkg3,dkg3;

int avg,diff1,diff2,diff3,diff4;
void setup()
{

```

```

Serial.begin(9600);
Serial.print("setup 1 --  ; ");
pinMode(buzzer,OUTPUT);
digitalWrite(buzzer,LOW);
lcd.begin();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("OVER LOAD          "); // You can make
spaces using well... spaces
lcd.setCursor(0, 1); // Or setting the cursor in the
desired position.
lcd.print("INDICATION    ");

setupfirstloadcell ();
setupsecondloadcell();
setupthirdloadcell();
setupfourloadcell();

Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);
lcd.clear();
}

void loop()
{
Blynk.run();
gpsread();

loopfirstloadcell ();

```

```

loopsecondloadcell();
loopthreeloadcell();
loopfourloadcell();

//kg1=kg1-5;
//kg2=kg2-3;
//kg3=kg3+9;
//kg4=kg4+24;


lcd.setCursor(0,0);
lcd.print("1:");
if(kg1<=9){lcd.print("000"); lcd.print(kg1);}
else if(kg1<=99){lcd.print("00"); lcd.print(kg1);
delay(400);}
else if(kg1<=999){lcd.print("0"); lcd.print(kg1);
delay(400);}
else if(kg1<=9999){lcd.print(""); lcd.print(kg1);
delay(400);}


lcd.setCursor(10,0);
lcd.print("2:");
if(kg2<=9){lcd.print("000"); lcd.print(kg2);}
else if(kg2<=99){lcd.print("00"); lcd.print(kg2);
delay(400);}
else if(kg2<=999){lcd.print("0"); lcd.print(kg2);
delay(400);}
else if(kg2<=9999){lcd.print(""); lcd.print(kg2);
delay(400);}

```

```

lcd.setCursor(0,1);
lcd.print("3:");
if(kg3<=9){lcd.print("000"); lcd.print(kg3);}
else if(kg3<=99){lcd.print("00"); lcd.print(kg3);
delay(400);}
else if(kg3<=999){lcd.print("0"); lcd.print(kg3);
delay(400);}
else if(kg3<=9999){lcd.print(""); lcd.print(kg3);
delay(400);}

lcd.setCursor(10,1);
lcd.print("4:");
if(kg4<=9){lcd.print("000"); lcd.print(kg4);}
else if(kg4<=99){lcd.print("00"); lcd.print(kg4);
delay(400);}
else if(kg4<=999){lcd.print("0"); lcd.print(kg4);
delay(400);}
else if(kg4<=9999){lcd.print(""); lcd.print(kg4);
delay(400);}
total_kg=kg1+kg2+kg3+kg4;

Serial.print("total_kg : ");
Serial.println(total_kg);

float leftWeight = kg1 + kg2;
float rightWeight = kg3 + kg4;

```

```

float diff = abs(leftWeight - rightWeight);
avg=total_kg/4;
avg=avg+100;
avg=avg-60;
Serial.print("avg :  ");
Serial.println(avg);

diff1 =abs(avg - kg1);
diff2 =abs(avg - kg2);
diff3 =abs(avg - kg3);
diff4 =abs(avg - kg4);

Serial.print("diff1----- : 
");
Serial.println(diff1);

Serial.print("diff2----- : 
");
Serial.println(diff2);

Serial.print("diff3----- : 
");
Serial.println(diff3);

Serial.print("diff4----- : 
");
Serial.println(diff4);

// if (diff < 50) { // Tolerance level in grams

```

```

//      Serial.println("Status: Balanced");
//  } else {
//      if (leftWeight > rightWeight) {
//          Serial.println("Status: Unbalanced - Heavier
on Left");
//      } else {
//          Serial.println("Status: Unbalanced - Heavier
on Right");
//      }
//  }

if(total_kg>=2000)
{
    lcd.setCursor(0,0);
    lcd.print("OVER LOAD      ");
    lcd.setCursor(0,1);
    lcd.print("                ");
    Serial.print("OVER LOAD -----
-- ");
    digitalWrite(buzzer,HIGH);
    delay(2000);
    lcd.clear();
}

else
if(diff1>=130||diff2>=130||diff3>=130||diff4>=130)
{
    lcd.setCursor(0,0);
    lcd.print("UNBALANCE      ");

```

```

    lcd.setCursor(0,1);
    lcd.print("                ");
    Serial.print("UNBALANCE -----
- ");
    digitalWrite(buzzer,HIGH);
    delay(2000);
    lcd.clear();
}
else
{
    digitalWrite(buzzer,LOW);

}
if (buttonread == 1)
{
    if(total_kg>=2000)
    {
        Blynk.logEvent("msg","OVER LOAD ");
        Blynk.virtualWrite(V6, "OVER LOAD ");
    }
    else
if(diff1>=130||diff2>=130||diff3>=130||diff4>=130)
    {
        Blynk.logEvent("msg","UNBALANCE");
        Blynk.virtualWrite(V6, "UNBALANCE ");
    }
    else
    {

```

```

        Blynk.virtualWrite(V6, "
    ");
    }
    Blynk.virtualWrite(V0, kg1);
    Blynk.virtualWrite(V1, kg2);
    Blynk.virtualWrite(V2, kg3);
    Blynk.virtualWrite(V3, kg4);
    Blynk.virtualWrite(V4, s);
    Blynk.virtualWrite(V7, total_kg);
    Blynk.virtualWrite(V8, avg);
    Serial.println("sensor value send blynk ");
}

}

BLYNK_WRITE(V5)
{
    int button = param.asInt();
    Serial.print("robot Button value :");
    Serial.println(button);
    buttonread = button;
}

void setupfirstloadcell ()
{
    scale1.begin(dout1, sck1);
    scale1.read();
    scale1.read_average(20);
    scale1.get_value(5);
    (scale1.get_units(5), 1);
    scale1.set_scale(2280.f);
}

```

```

    scale1.tare();
    scale1.read();
    scale1.read_average(20);
    scale1.get_value(5);
    (scale1.get_units(5), 1);

}

void loopfirstloadcell ()
{
    (scale1.get_units(), 1);
    p1 = scale1.get_units(10);
    Serial.print("p1:");
    Serial.println(p1);
    (p1, 1);
    if (p1<=0) {p1=p1*(-1);      Serial.println("positive
conversion");}
    kg1=(p1*4.67);

    if (kg1<0.1) {kg1=0;}
    scale1.power_down();      // put the ADC
in sleep mode
    delay(1000);
    scale1.power_up();
    if (kg1<=5)
    {
        kg1=0;
    }

    Serial.print("kg1 : ");

```

```

        Serial.println(kg1);

        delay(100);

    }
void setupsecondloadcell()
{
    scale2.begin(dout12, sck2);
    scale2.read();
    scale2.read_average(20);
    scale2.get_value(5);
    (scale2.get_units(5), 1);
    scale2.set_scale(2280.f);
    scale2.tare();
    scale2.read();
    scale2.read_average(20);
    scale2.get_value(5);
    (scale2.get_units(5), 1);
}
void loopsecondloadcell()
{

    (scale2.get_units(), 1);
    p2 = scale2.get_units(10);
    Serial.print("p2:");
    Serial.println(p2);
    (p2, 1);
    if(p2<=0){p2=p2*(-1);    Serial.println("positive
conversion");}

```

```

    kg2=(p2*4.44) ;

    if(kg2<0.1){kg2=0;}
        scale2.power_down();           // put the ADC
in sleep mode
        delay(1000);
        scale2.power_up();
if(kg2<=5)
{
    kg2=0;
}

    Serial.print("kg2 : ");
    Serial.println(kg2);
    delay(100);
}

void setupthreeloadcell()
{
    scale3.begin(dout3, sck3);
    scale3.read();
    scale3.read_average(20);
    scale3.get_value(5);
    (scale3.get_units(5), 1);
    scale3.set_scale(2280.f);
    scale3.tare();
    scale3.read();
    scale3.read_average(20);
    scale3.get_value(5);
    (scale3.get_units(5), 1);

```

```

}
void loopthreeloadcell()
{
    (scale3.get_units(), 1);
    p3 = scale3.get_units(10);
    Serial.print("p3:");
    Serial.println(p3);
    (p3, 1);
    if (p3<=0) {p3=p3*(-1);      Serial.println("positive
conversion");}
    kg3=(p3*4.54);

    if (kg3<0.1) {kg3=0;}
    scale3.power_down();        // put the ADC
in sleep mode
    delay(1000);
    scale3.power_up();
    if (kg3<=5)
    {
        kg3=0;
    }

    Serial.print("kg3 : ");
    Serial.println(kg3);

    delay(100);
}

```

```

void setupfourloadcell()
{
    scale4.begin(dout4, sck4);
    scale4.read();
    scale4.read_average(20);
    scale4.get_value(5);
    (scale4.get_units(5), 1);
    scale4.set_scale(2280.f);
    scale4.tare();
    scale4.read();
    scale4.read_average(20);
    scale4.get_value(5);
    (scale4.get_units(5), 1);
}

void loopfourloadcell()
{
    (scale4.get_units(), 1);
    p4 = scale4.get_units(10);
    Serial.print("p4:");
    Serial.println(p4);
    (p4, 1);
    if(p4<=0){p4=p4*(-1);      Serial.println("positive
conversion");}
    kg4=(p4*4.50);

    if(kg4<0.1){kg4=0;}
    scale4.power_down();      // put the ADC
in sleep mode
    delay(1000);
}

```

```

        scale4.power_up();
if (kg4<=5)
{
    kg4=0;
}

    Serial.print("kg4 : ");
    Serial.println(kg4);

    delay(100);
}

void gpsread()
{
    smartDelay(200); /* Generate precise delay of
1ms */

    unsigned long start;

    lat_val = gps.location.lat(); /* Get latitude
data */

    loc_valid = gps.location.isValid(); /* Check
if valid location data is available */

    lng_val = gps.location.lng(); /* Get
longitude data */

    alt_m_val = gps.altitude.meters(); /* Get
altitude data in meters */

    alt_valid = gps.altitude.isValid(); /* Check
if valid altitude data is available */

    hr_val = gps.time.hour(); /* Get hour */
    min_val = gps.time.minute(); /* Get minutes
*/

```

```

        sec_val = gps.time.second();  /* Get seconds
*/

        time_valid = gps.time.isValid();  /* Check if
valid time data is available */

        if (!loc_valid)
        {
            Serial.print("Latitude : ");
            Serial.println("*");
            Serial.print("Longitude : ");
            Serial.println("*");
        }
        else
        {
            DegMinSec(lat_val);

            DegMinSec(lng_val); /* Convert the decimal
degree value into degrees minutes seconds form */

            longn=(lng_val, 6);

        }
        if (!alt_valid)
        {

        }
        else
        {

```

```

    }
    if (!time_valid)
    {

    }
    else
    {
        char time_string[32];
        sprintf(time_string, "Time : %02d/%02d/%02d
\n", hr_val, min_val, sec_val);
        Serial.print(time_string);
    }

if(!alt_valid){lat_val=11.017301;lng_val=76.9347119;
}

s = "www.google.com/maps/dir/";
s += String(lat_val, 6);
s += ",";
s += String(lng_val, 6);
s += "/";

Blynk.virtualWrite(V4,s);
}

static void smartDelay(unsigned long ms)
{
    unsigned long start = millis();
    do

```

```

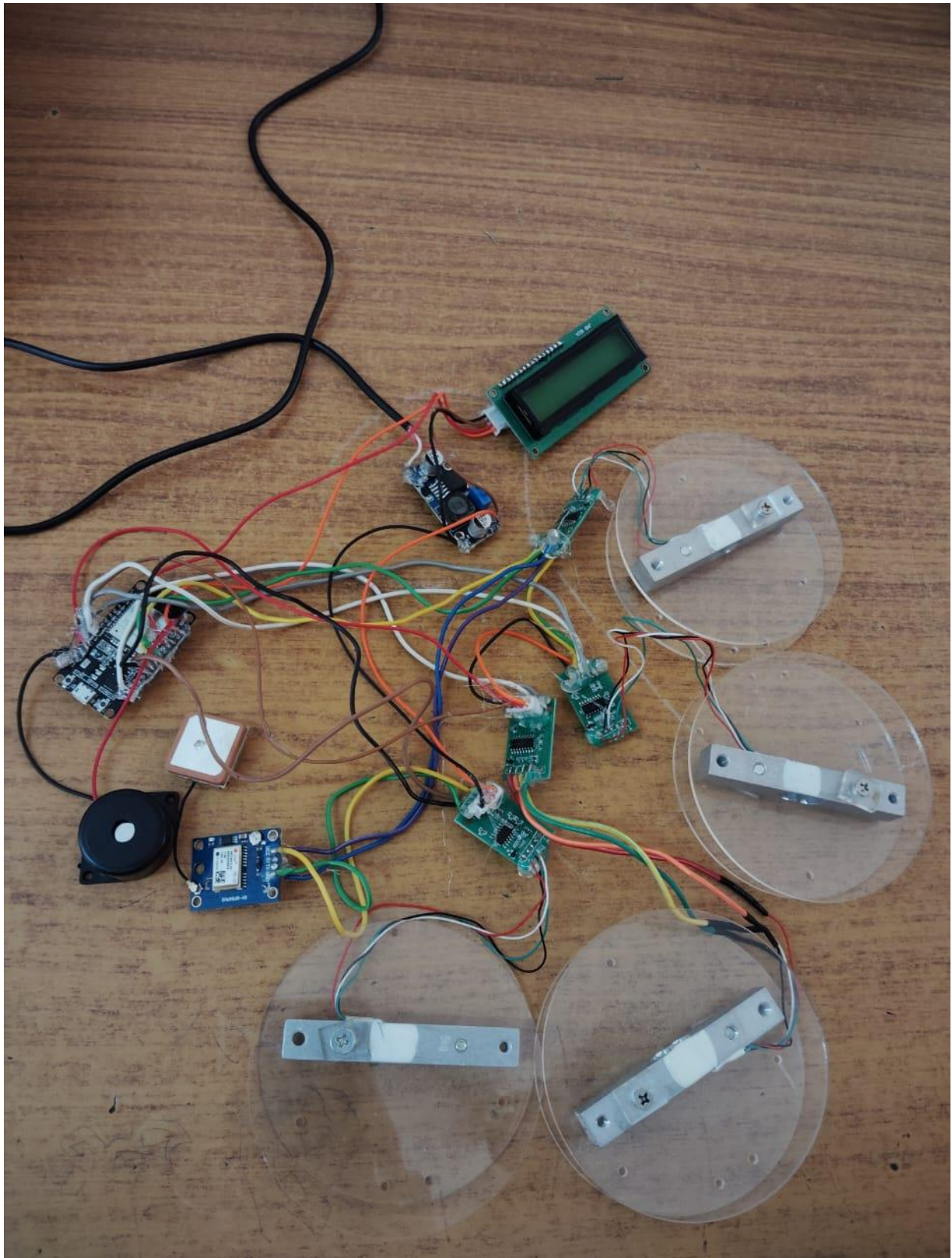
{
    while (Serial.available()) /* Encode data read
from GPS while data is available on serial port */
        gps.encode(Serial.read());
/* Encode basically is used to parse the string
received by the GPS and to store it in a buffer so
that information can be extracted from it */
    } while (millis() - start < ms);
}

void DegMinSec( double tot_val) /* Convert data in
decimal degrees into degrees minutes seconds form */
{
    degree = (int)tot_val;
    minutes = tot_val - degree;
    seconds = 60 * minutes;
    minutes = (int)seconds;
    mins = (int)minutes;
    seconds = seconds - minutes;
    seconds = 60 * seconds;
    secs = (int)seconds;
}

```

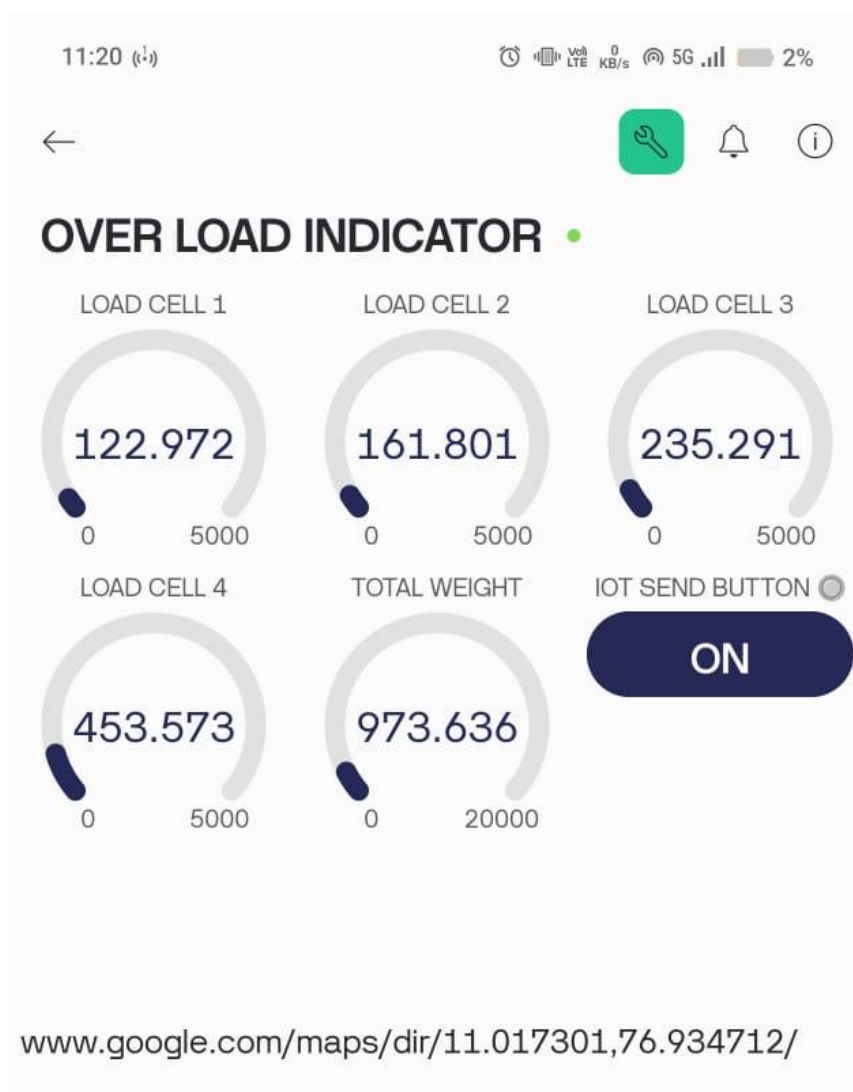
## 5. EXPERIMENTAL RESULTS

### 5.1 HARDWARE SETUP



## 5.1. OUTPUT





## **5.2. OUTPUT DESCRIPTION**

The mobile app is an integral component of the system, providing remote viewing of real-time data gathered from the truck. It is created with the use of the Blynk IoT platform, which supports real-time visualization, alerting, and control through smartphone.

### **Load Cell Display**

The application presents separate readings from four load cells, which are each mounted at a respective corner of the cargo platform. This configuration provides proper weight distribution analysis.

The values indicated for every load cell are:

- Load Cell 1: 122.972 units
- Load Cell 2: 161.801 units
- Load Cell 3: 235.291 units
- Load Cell 4: 453.573 units

Every load cell is equipped with a gauge that has a range of 0 to 5000 units, giving a visual indication of unbalanced or uneven loading. Individual readings are monitored to detect localized overloading or cargo shift in transit.

### **Total Weight Indicator**

The overall weight on the vehicle is determined by adding all four readings of the load cells. In the data presented, the total weight is 973.636 units.

This figure is displayed on a larger gauge with a scale of 0 to 20000 units, enabling a greater variety of cargo weights to be easily imagined. This facility enables the user to make rapid judgement as to whether the aggregate weight is within legal and safe boundaries.

## **IoT Send Button**

A prominent switch labeled "IOT SEND BUTTON" enables or disables data transmission from the ESP32 to the cloud. When the switch is ON, the microcontroller continuously sends live readings from the load cells and GPS to the Blynk cloud server.

This control feature is especially useful during testing, troubleshooting, or maintenance. It prevents unnecessary data uploads and conserves system resources when not in operation.

## **GPS Location Tracking**

The system incorporates a NEO-6M GPS module, which records the coordinates of the truck at the present moment. These are presented as a clickable Google Maps link in the app.

### **Example:**

<https://www.google.com/maps/dir/11.017301,76.934712/>

Opening the link loads Google Maps and indicates the location of the vehicle on the map in real time. This is very helpful in fleet tracking, route planning, and response to incidents during overloading or mismanagement of cargo.

## **Connectivity Status Indicator**

A green dot at the top of the screen indicates that the system is live connected to the Blynk cloud server. This live status of connection ensures the user knows if the device is online and working fine.

If the dot is red or grey, the ESP32 is not connected to Wi-Fi or the cloud, and the user needs to take corrective action.

### **Instant Alerts and Remote Monitoring**

When the combined weight goes over a predetermined limit (e.g., 1000 units), the system automatically triggers push notifications through the Blynk app. The alert system supplants the use of SMS and guarantees that the driver or manager is notified instantly, even without SIM balance or mobile signal.

The mobile app serves as both monitoring tool and alert system, guaranteeing secure and compliant cargo transportation.

## **6. CONCLUSION AND FUTURE SCOPE**

### **6.1. CONCLUSION**

The logistics and transportation sectors are increasingly confronted with the challenge of safely and legally managing truck weights. Conventional weight measurement technologies like Static Weighing Systems, Weigh-In-Motion (WIM) Systems, and LiDAR-Based Load Volume Measurement have been extensively used to address this challenge. Nevertheless, every one of these proposals has certain critical drawbacks in terms of accuracy, scalability, cost, and operational efficiency.

Static Weighing Systems are trustworthy but necessitate stopping trucks at designated weigh stations, leading to delay, rising operational expenses, and restricting their usage to individual points. They are inconvenient and not designed for ongoing or real-time tracking, hence unsuitable for contemporary logistics calling for constant surveillance. Analogously, Weigh-In-Motion (WIM) Systems, that are real-time motion-based truck weight estimators, are founded on road condition, speed of vehicle, and sensor technology employed. They are expensive to install since they entail the employment of embedded sensors in the road, and they may be repaired at places such as highway portions or toll roads only, limiting their availability. Furthermore, the accuracy of the WIM system depends on the suspension, tire pressure, and speed of the vehicle, thus posing the problem of attaining precise real-time data along various routes.

LiDAR-Based Load Volume Measurement is a different alternative that utilizes 3D laser sensors to estimate load volume and calculate weight based on material density. However, the use of precise material density in calculating weight introduces a substantial source of error, particularly when working with freight of disparate shapes or densities. Further, LiDAR technology is costly, and its precision may be threatened by environmental factors like dust, weather, or sensor calibration. This renders them less universal for various applications and

more suitable for particular applications.

In order to overcome these drawbacks, our suggested vehicle-mounted weight monitoring system is a more efficient and viable alternative. The system is intended for real-time use, providing constant weight monitoring without the need for stopping trucks or using infrastructure laid on the road. Since sensors are mounted on the vehicle itself, the system is not only portable but can also be used with a variety of vehicles and routes. It is able to provide precise weight readings irrespective of environmental conditions like road condition or vehicle speed. Further, the system is economical as it minimizes the use of expensive road infrastructure and is scalable across different fleet sizes and operating environments.

Finally, the proposed system provides a major improvement compared to other weight monitoring technologies. It measures weight accurately and in real time without any boundaries of location, road surface, or infrastructure cost. Through augmented compliance with regulation loads, preventing overloading, and lower operational costs, our system promotes road safety and the possibility of optimization of logistics. It is a cost-saving and future-oriented solution for the emerging transportation industry which opens the path to intelligent, safe, and efficient fleet management.

## **6.2. FUTURE SCOPE**

The envisaged vehicle-mounted system for weight monitoring has great scope for growth and enhancement in many directions. One of the most important future developments is integration with telematics systems to monitor vehicle performance in real-time, in addition to weight. This might assist fleet managers in optimizing routes, fuel consumption, and overall operational efficiency. Increased developments in sensor technology would improve the accuracy of the system. Through the integration of more sensors like tire pressure, suspension condition, and load distribution, the system would be able to offer more accurate

weight measurements. Machine learning algorithms would also be applied to process sensor information, allowing for predictive maintenance and the identification of performance degradation prior to failure.

A further thrilling development is the application of cloud-based analytics of data. This would allow fleet operators to remotely access real-time data, monitor long-term trends, and make informed decisions based on data. Cloud platforms would also facilitate the reporting of automated regulatory compliance, making it easy to ensure legal weight limits compliance.

Future integration of predictive analytics and AI has the potential to further upgrade the system's capability to recognize potential overloading ahead of time, sending pre-emptive warnings and enabling more efficient route management. Global scalability will also enable the system to accommodate various transportation regulations and infrastructure for international application.

Lastly, there is scope for creating more efficient and sustainable options by using renewable energy sources, like solar energy, to power the system. These developments will not only enhance the performance of the system but also make the transportation system safer and more sustainable.

## REFERENCES

- [1] Arrive Alive, “Overloading and Road Safety,” pp.1–5,2016, [Online]. Available: <https://arrivealive.co.za/pages.aspx?u=Overloading-and-Road-Safety>.
- [2] R. Shah, Y. Sharma, B. Mathew, V. Kateshiya, and J. Parmar, “Review Paper on Overloading Effect,” *Int. J. Adv. Sci. Res. Manag.*, vol. 1, no. 4, pp. 2–5, 2016, [Online]. Available: [www.ijasrm.com](http://www.ijasrm.com).
- [3] T. B. JOEWONO and H. KUBOTA, “Safety and Security Improvement in Public Transportation Based on Public Perception in Developing Countries,” *IATSS Res.*, vol. 30, no. 1, pp. 86–100, 2006, doi: 10.1016/s0386-1112(14)60159-x.
- [4] K. Hassan, A. Sam, and D. Machuve, “OVERVIEW ON PASSENGERS OVERLOAD CONTROL IN PUBLIC BUSES CASE STUDY: TANZANIA,” vol. 2, no. 8, pp. 2536–2540, 2013.
- [5] S. Xu and Q. Zhao, “Study on the vehicle-mounted overloading control system for passenger vehicles,” *Procedia Eng.*, vol. 15, pp. 1214–1218, 2011, doi: 10.1016/j.proeng.2011.08.224.
- [6] S. Hu, M. Kong, and C. She, “Design of vehicle overload detection system based on geophone,” *J. Phys. Conf. Ser.*, vol. 887, no. 1, 2017, doi: 10.1088/1742-6596/887/1/012021.
- [7] M. G. Guillemette, I. Fontaine, and C. Caron, “Hybrid RFID-GPS real-time location system for human resources: Development, impacts and perspectives,”

Proc. Annu. Hawaii Int. Conf. Syst. Sci., pp. 1–10, 2008, doi: 10.1109/HICSS.2008.195.

[8] K. R. Prasanna and M. Hemalatha, “RFID GPS and GSM based logistics vehicle load balancing and tracking mechanism,” *Procedia Eng.*, vol. 30, no. 2011, pp. 726–729, 2012, doi